

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA

XANIM'S VAULT

Sistema automatizado de alimentação para felinos

Relatório final apresentado à disciplina de
DIM0616 - Sistemas Embarcados,
Docente Mônica Magalhães Pereira.

Andriel Vinicius de Medeiros Fernandes
Lucas Vinicius Dantas de Medeiros
María Paz Marcato

Natal/RN 2026

Sumário

1	Introdução	4
2	Descrição do sistema	4
2.1	Objetivo	4
2.2	Usuário	4
2.3	Visão geral da arquitetura	4
2.4	Entradas, processamento e saídas	5
2.5	Cálculo do plano alimentar	5
2.6	Escopo do protótipo	6
2.7	Fora do escopo	6
3	Estado da arte	6
4	Requisitos funcionais e não funcionais	7
4.1	Requisitos funcionais	7
4.1.1	RF01 - Cadastro do animal	7
4.1.2	RF02 - Geração do plano alimentar	7
4.1.3	RF03 - Configuração e distribuição das refeições	8
4.1.4	RF04 - Operação offline	8
4.1.5	RF05 - Atualização de dados	8
4.1.6	RF06 - Liberação e controle da ração	8
4.1.7	RF07 - Identificação NFC	8
4.1.8	RF08 - Monitoramento do consumo	8
4.1.9	RF09 - Visualização de estatísticas	8
4.1.10	RF10 - Registro de Eventos	8
4.2	Requisitos não funcionais	8
4.2.1	RNF01 - Persistência local	8
4.2.2	RNF02 - Confiabilidade após queda de energia	8
4.2.3	RNF03 - Precisão na liberação	9
4.2.4	RNF04 - Tempo de resposta do bloqueio	9
4.2.5	RNF05 - Comunicação	9
4.2.6	RNF06 - Tempo de detecção NFC	9
4.3	Regras de sistema	9
5	Modelagem	9
5.1	Diagrama de componentes	10
5.2	Diagrama de blocos do hardware	10
5.3	Diagrama de fiação	11
5.4	Máquina de estados	11
5.4.1	Parte 1: Cadastro e identificação inicial	12
5.4.2	Parte 2: Detecção de NFC e tratamento de intrusos	13
5.4.3	Parte 3: Liberação e monitoramento de consumo	14
5.4.4	Parte 4: Atualização de configurações	14
5.5	Modelo de dados	15
6	Implementação	16
6.1	Tecnologias utilizadas	16
6.1.1	Hardware:	16
6.1.2	Software:	16
6.2	Tópicos MQTT	16

6.2.1	CAT/NFC	16
6.2.2	CAT/SYNC	16
6.2.3	CAT/INTRUDER	17
6.2.4	FEED/REGISTER	17
6.3	Requisitos implementados	17
6.3.1	RF01 - Cadastro do animal	17
6.3.2	RF02 - Geração do plano alimentar	17
6.3.3	RF03 - Configuração e distribuição das refeições	17
6.3.4	RF05 - Atualização de dados	17
6.3.5	RF07 - Identificação NFC	18
6.3.6	RF08 - Monitoramento do consumo (Parcial)	18
6.3.7	RF09 - Visualização de estatísticas	18
6.3.8	RF10 - Registro de Eventos	18
6.3.9	RNF01 - Persistência local	18
6.3.10	RNF02 - Confiabilidade após queda de energia	18
6.3.11	RNF04 - Tempo de resposta do bloqueio	18
6.3.12	RNF05 - Comunicação	18
6.3.13	RNF06 - Tempo de detecção NFC	18
6.4	Limitações	18
6.4.1	RF04 - Operação offline	18
6.4.2	RF06 - Liberação e controle da ração (parcial)	19
6.4.3	RNF03 - Precisão na liberação	19
6.5	Dificuldades encontradas	19
6.5.1	Reativação do WiFi/MQTT e risco de travamento	19
6.5.2	Integração da célula de carga (HX711)	19
6.5.3	Configuração dinâmica dos horários	19
6.5.4	Centralização MQTT no backend	19
6.5.5	Configuração do Mosquitto em Docker	19
6.5.6	Orquestração de containers	19
6.6	Uso de Inteligência Artificial Generativa	20
7	Conclusão	20
7.1	Resultados alcançados	20
7.2	Limitações	21
7.3	Melhorias futuras	21
8	Referências	22

1 Introdução

A alimentação adequada dos felinos é fundamental para o seu desenvolvimento saudável. Em residências com mais de um gato em condições distintas como por exemplo, animais castrados e não castrados, filhotes ou gatos em diferentes fases etárias, torna necessário oferecer porções e tipos de ração específicos para cada animal, uma vez que as necessidades nutricionais variam consideravelmente entre eles. Gatos castrados, por exemplo, tendem a ter metabolismo mais lento e exigem dietas com menor teor calórico em comparação aos não castrados.

Entretanto, o comportamento natural dos felinos dificulta o controle alimentar individualizado. Por serem animais independentes e territorialistas, é comum que um gato consuma a ração destinada a outro, especialmente quando este demora mais para se alimentar ou se ausenta momentaneamente do pote. Esse consumo indevido de alimento gera desequilíbrios nutricionais que podem comprometer a saúde dos animais ao longo do tempo.

Além disso, gatos devem se alimentar em múltiplos horários ao longo do dia, de modo a evitar tanto o acúmulo de calorias desnecessário quanto a ansiedade associada à concentração da alimentação em apenas um ou dois momentos do dia. Soluções existentes no mercado, como comedouros automáticos com horário programado, apresentam limitações relevantes: a liberação de ração em horários fixos, sem qualquer mecanismo de controle de acesso, não impede que outro animal da casa consuma a porção alheia, e o esquecimento humano de repor a ração manualmente pode resultar em períodos de privação alimentar.

Diante desse cenário, o presente projeto propõe o desenvolvimento do Xanim's Vault, um dispenser inteligente e automático de ração, capaz de identificar o animal por meio de tecnologia NFC, calcular e liberar porções individualizadas ao longo do dia e impedir o acesso de outros felinos à ração de um gato específico.

2 Descrição do sistema

2.1 Objetivo

O sistema tem como objetivo gerenciar a dieta de cada felino cadastrado de forma automatizada e inteligente, atuando no cálculo das porções diárias, na liberação da ração nos horários programados e no mecanismo de defesa contra o consumo por animais não autorizados.

2.2 Usuário

O principal usuário do sistema é o pai/mãe de pet, preocupado com a saúde alimentar do seu gato e com a possibilidade de outros animais da casa interferirem em sua dieta.

2.3 Visão geral da arquitetura

O Xanim's Vault é composto por quatro grandes blocos, que se comunicam conforme ilustrado no diagrama de componentes (ver Seção 5.1):

- **Aplicativo (UI)**: aplicação para cadastro do animal e visualização de estatísticas de consumo, desenvolvida em Next.js com Material UI;
- **Backend / CMS (Strapi)**: responsável por armazenar os dados de gatos, dietas, horários e registros, além de calcular o plano alimentar;
- **Banco de dados (SQLite)**: banco de dados leve utilizado pelo Strapi para persistência das informações;
- **Broker MQTT (Mosquitto)**: intermediário de comunicação publish/subscribe entre o backend e o sistema embarcado;
- **Sistema embarcado (ESP8266)**: responsável pela leitura de NFC, pesagem da ração, liberação das porções, bloqueio do pote e envio de eventos.

A comunicação entre os módulos ocorre da seguinte forma:

- **Aplicativo ↔ Backend**: protocolo HTTP/REST;
- **Backend ↔ Sistema embarcado**: protocolo MQTT, via broker Mosquitto.

2.4 Entradas, processamento e saídas

Entradas do sistema:

1. Cadastro do animal (nome, peso, data de nascimento, status de castração e identificador NFC);
2. Leitura do tempo corrente, via módulo RTC;
3. Detecção da tag NFC e leitura da célula de carga (balança).

Processamento:

1. Criação do plano alimentar pelo backend (CMS), a partir dos dados cadastrais do gato;
2. Processos do sistema embarcado: alerta sonoro (buzzer), liberação da ração e monitoramento de consumo.

Saídas do sistema:

1. Abertura/fechamento do pote de ração;
2. Envio de estatísticas e eventos ao servidor.

2.5 Cálculo do plano alimentar

O cálculo da quantidade diária de ração segue as diretrizes nutricionais da AAHA (2021 AAHA Nutrition and Weight Management Guidelines for Dogs and Cats), utilizando as fórmulas de necessidade energética em repouso (RER) e necessidade energética de manutenção (MER):

$$\text{RER} = \text{peso}_{\text{kg}}^{0.75} \times 70$$

$$\text{MER} = \text{RER} \times \text{fator do estágio de vida}$$

O fator do estágio de vida varia conforme a condição do animal (por exemplo, entre 1,2 e 1,4 para adultos castrados, e entre 1,4 e 1,6 para adultos intactos). A partir do valor de MER (em kcal/dia), o sistema converte a energia necessária em gramas de ração, considerando a concentração calórica da ração de referência (Quatree Gourmet):

- Filhote: 3820 kcal/kg (3,82 kcal/g);
- Adulto: 3800 kcal/kg (3,80 kcal/g);
- Adulto castrado: 3705 kcal/kg (3,70 kcal/g).

A quantidade diária calculada é então dividida entre os horários de alimentação configurados (3, 4 ou 6 horários, sendo 4 o padrão). Durante a liberação, a ração é dispensada aos poucos, permitindo que o ESP8266 monitore o peso na balança em tempo real e interrompa a liberação assim que a porção programada for atingida, com margem de erro de ± 5 g.

2.6 Escopo do protótipo

Para a entrega ao final do semestre, foi definido que a simulação seria feita de forma física, mas não necessariamente com todos os componentes em condição de produção:

- O servidor web (CMS) roda localmente, utilizando Strapi;
- A “coleira” do gato é representada apenas pelo chip NFC;
- Elementos de papelão são utilizados para representar potes e travas.

2.7 Fora do escopo

Os seguintes itens foram explicitamente definidos como fora do escopo do projeto:

- Reconhecimento facial de animais e uso de câmeras;
- Integração com serviços externos;
- Controle simultâneo de múltiplos gatos ou múltiplos potes;
- Uso de inteligência artificial;
- Controle de estoque de ração;
- Aplicativo comercial / sistema em produção;
- Atualização de firmware remota.

3 Estado da arte

Durante a etapa de pesquisa, foram identificados diversos trabalhos e produtos relacionados a alimentadores automáticos de animais, que serviram de referência para a definição do escopo e das funcionalidades do Xanim's Vault:

Trabalho / Produto	Descrição
Comedouro automático para cães em situação de rua	Projeto voltado ao apoio de ONGs no cuidado de cães resgatados, com liberação automatizada de ração.
Alimentador automático para nutrição de gatos com ESP32 (UFRGS)	Trabalho de conclusão que utiliza ESP32 para controle da alimentação de felinos.
ComunaPet - Solução inteligente para alimentação de pets via IoT	Bebedouro automatizado e comedouro manual para ani-

	mais de rua, com monitoramento via aplicativo PetFriendly.
Dispenser de ração automático com Arduino	Projeto de eletrônica com liberação automatizada de ração utilizando Arduino.
Alimentador com ESP32, MQTT e dashboards (UFERSA)	Sistema embarcado que utiliza MQTT para comunicação e dashboards para visualização dos dados de alimentação.
Coleira com tecnologia NFC	Trabalho relacionado ao uso de tags NFC para identificação individual de animais.

Além desses trabalhos acadêmicos, foram analisados produtos comerciais, como comedouros automáticos com conectividade Wi-Fi e controle via aplicativo (ex.: linha Petlibro) e comedouros/bebedouros duplos de baixo custo. Em geral, esses produtos comerciais permitem programação de horários e porções, mas não oferecem identificação individual do animal nem mecanismo de bloqueio de acesso para outros pets da casa, diferencial central proposto pelo Xanim's Vault.

4 Requisitos funcionais e não funcionais

Nesta seção são apresentados os requisitos finais do sistema, já refinados a partir da proposta inicial.

4.1 Requisitos funcionais

4.1.1 RF01 - Cadastro do animal

O sistema deve permitir o cadastro de um gato contendo as informações de nome, peso, data de nascimento, status de castração e identificador NFC;

4.1.2 RF02 - Geração do plano alimentar

O sistema calcula automaticamente a quantidade diária de ração com base nas características cadastradas do gato, considerando a quantidade de refeições configurada pelo usuário (3, 4 ou 6 por dia).

4.1.3 RF03 - Configuração e distribuição das refeições

O sistema permite configurar 3, 4 ou 6 refeições por dia, com definição manual dos horários de alimentação, e distribui automaticamente a quantidade diária de ração entre os horários configurados.

4.1.4 RF04 - Operação offline

O sistema deve continuar operando utilizando dados salvos na memória quando estiver sem conexão com Internet.

4.1.5 RF05 - Atualização de dados

O embarcado deve atualizar os dados armazenados localmente sempre que novas configurações forem recebidas da API.

4.1.6 RF06 - Liberação e controle da ração

O sistema deve liberar automaticamente a porção programada nos horários definidos, junto de um alarme sonoro.

4.1.7 RF07 - Identificação NFC

O sistema deve detectar tags NFC posicionadas a até 4 cm do leitor e comparar a tag detectada com a cadastrada se houver. Caso seja válida, o acesso ao pote deve ser permitido; caso contrário, o pote deve ser bloqueado e um alerta enviado à API.

4.1.8 RF08 - Monitoramento do consumo

O sistema deve monitorar o consumo do gato a partir do peso da balança e enviar registros de consumo (ou ausência dele) 60 segundos antes do próximo horário de liberação. O consumo deve ser classificado como:

- Consumo total => até 10%
- Consumo parcial => 11% até 90%
- Não consumiu => acima de 90%

O consumo será registrado dessa forma para evitar variações na balança ao longo do tempo.

4.1.9 RF09 - Visualização de estatísticas

O sistema deve permitir visualizar: horários em que o gato se alimentou; consumo parcial ou completo; tentativas de acesso por outros gatos; alertas gerados pelo sistema.

4.1.10 RF10 - Registro de Eventos

O embarcado deve enviar à API: horário da alimentação; quantidade liberada; status do consumo; identificação NFC detectada; alertas de acesso inválido.

4.2 Requisitos não funcionais

4.2.1 RNF01 - Persistência local

Os dados devem persistir mesmo quando o sistema esteja offline.

4.2.2 RNF02 - Confiabilidade após queda de energia

Os dados armazenados na memória flash não devem ser perdidos em caso de queda de energia ou reinicialização inesperada.

4.2.3 RNF03 - Precisão na liberação

A liberação deve ser interrompida quando a quantidade programada for atingida com margem de erro de $\pm 5g$.

4.2.4 RNF04 - Tempo de resposta do bloqueio

Ao detectar aproximação indevida, o sistema irá fechar o pote em até 1 segundo e enviar um alertar à API.

4.2.5 RNF05 - Comunicação

A comunicação entre o servidor e o sistema embarcado deve utilizar o protocolo MQTT via broker Mosquitto e HTTP do servidor para o app WEB.

4.2.6 RNF06 - Tempo de detecção NFC

A tag NFC deve ser detectada em no máximo 1 segundo após a aproximação ao leitor.

4.3 Regras de sistema

- **RN01:** A soma das porções distribuídas ao longo do dia deve ser igual à quantidade diária calculada pelo CMS.
- **RN02:** Somente a tag NFC cadastrada para o gato pode acessar o seu pote de ração.
- **RN03:** A liberação da ração deve ocorrer apenas nos horários programados.
- **RN04:** Cada evento de alimentação deve gerar um registro armazenado no CMS.

5 Modelagem

Esta seção apresenta os diagramas elaborados durante a fase de projeto, com a respectiva explicação de cada um.

5.1 Diagrama de componentes

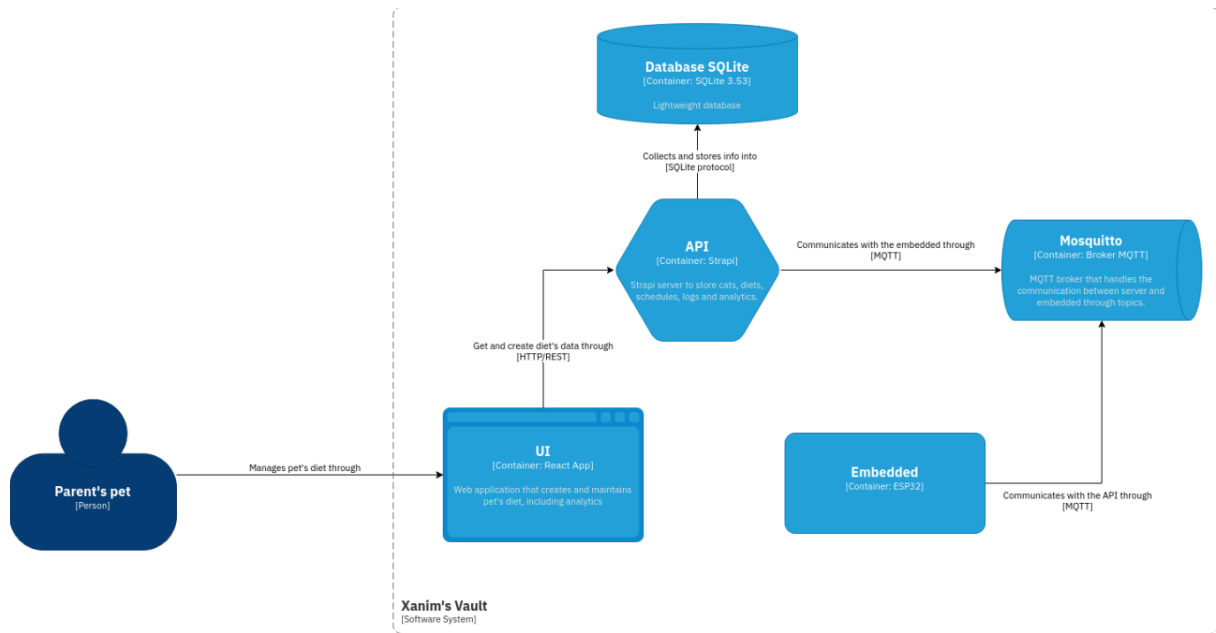


Figura 1: Diagrama de componentes do Xanim's Vault, no estilo C4, mostrando a interação entre o usuário (pai/mãe do pet), a aplicação (UI), o CMS (Strapi), o banco de dados (SQLite), o broker MQTT (Mosquitto) e o sistema embarcado (ESP32).

O diagrama de componentes descreve a arquitetura de alto nível do sistema. O usuário interage com a UI, que se comunica com o backend CMS via HTTP/REST para recuperar e atualizar os dados da dieta do animal. O CMS armazena as informações no banco SQLite e publica os dados de configuração para o sistema embarcado através do broker MQTT (Mosquitto). O sistema embarcado, por sua vez, envia ao broker os eventos de alimentação, consumo e alertas de intrusão, que são consumidos pelo CMS.

5.2 Diagrama de blocos do hardware

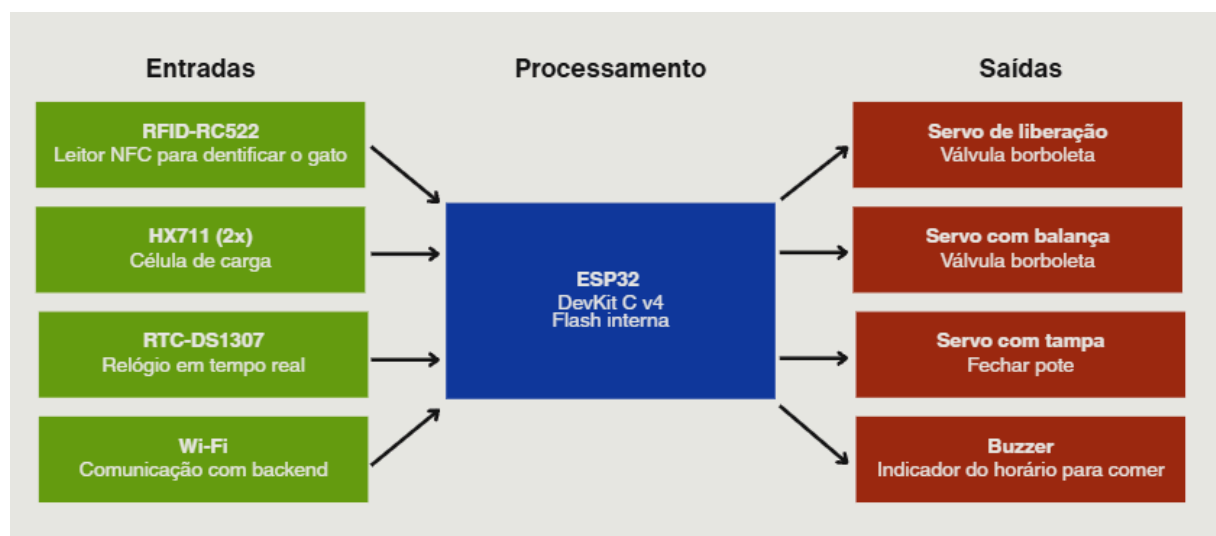


Figura 2: Diagrama de blocos do hardware, dividido em entradas, processamento e saídas.

O diagrama de blocos organiza o hardware em três grupos:

- **Entradas:** leitor NFC (RFID-RC522), duas células de carga (HX711), módulo RTC (DS1307/DS3231) e módulo Wi-Fi;
- **Processamento:** microcontrolador ESP32 DevKit C v4, com memória flash interna;
- **Saídas:** servo de liberação (válvula borboleta), servo com balança (válvula borboleta), servo com tampa (fechamento do pote) e buzzer (indicador sonoro do horário de alimentação).

5.3 Diagrama de fiação

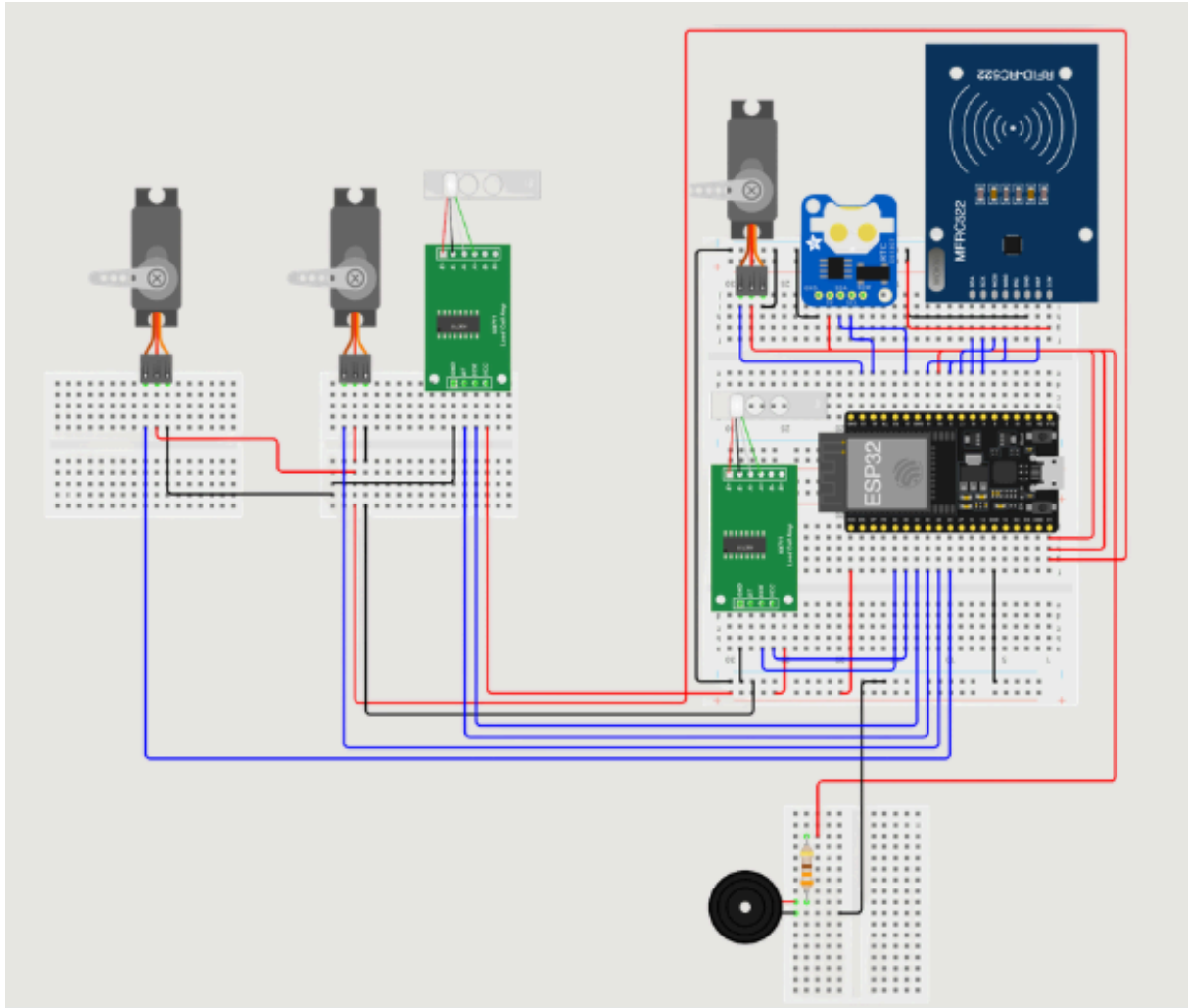


Figura 3: Diagrama de fiação da montagem em protoboard, mostrando a conexão do ESP32 aos servomotores, ao módulo RFID-RC522, às células de carga (via módulos HX711) e ao buzzer.

Este diagrama detalha as conexões físicas entre o ESP32 e os demais componentes: os servomotores responsáveis pela liberação da ração e pelo fechamento do pote, o módulo RFID-RC522 para leitura das tags NFC, os módulos HX711 conectados às células de carga, e o buzzer utilizado como indicador sonoro.

5.4 Máquina de estados

O comportamento do sistema embarcado foi modelado como uma máquina de estados dividida em quatro partes.

5.4.1 Parte 1: Cadastro e identificação inicial

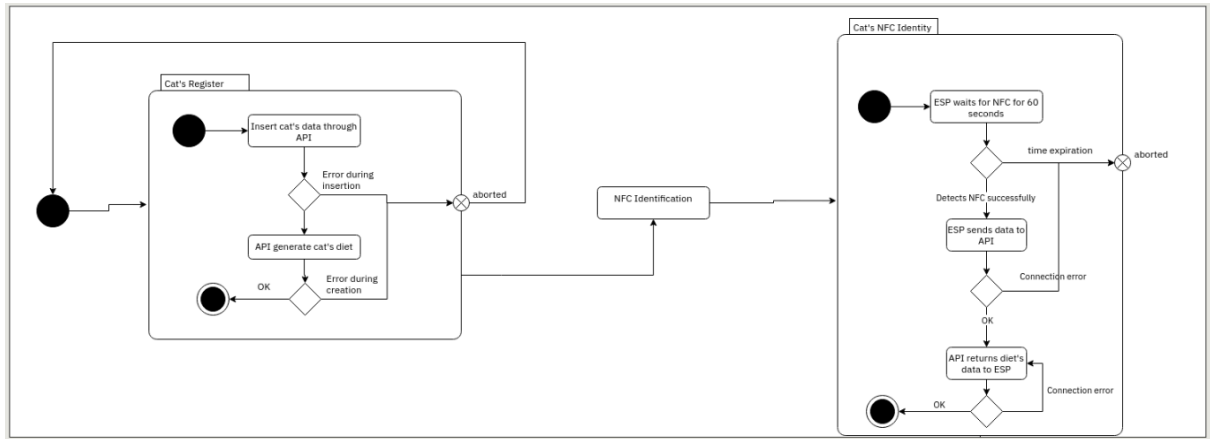


Figura 4: Etapa inicial: registro do gato, leitura de NFC e envio de dados ao embarcado.

Nesta primeira etapa, os dados do gato são inseridos via API. Caso não haja erro na inserção, o CMS gera automaticamente o plano alimentar. Em seguida, o ESP32 aguarda a detecção de um NFC por até 60 segundos; ao detectar, envia os dados à API, que retorna as informações da dieta para o embarcado.

5.4.2 Parte 2: Detecção de NFC e tratamento de intrusos

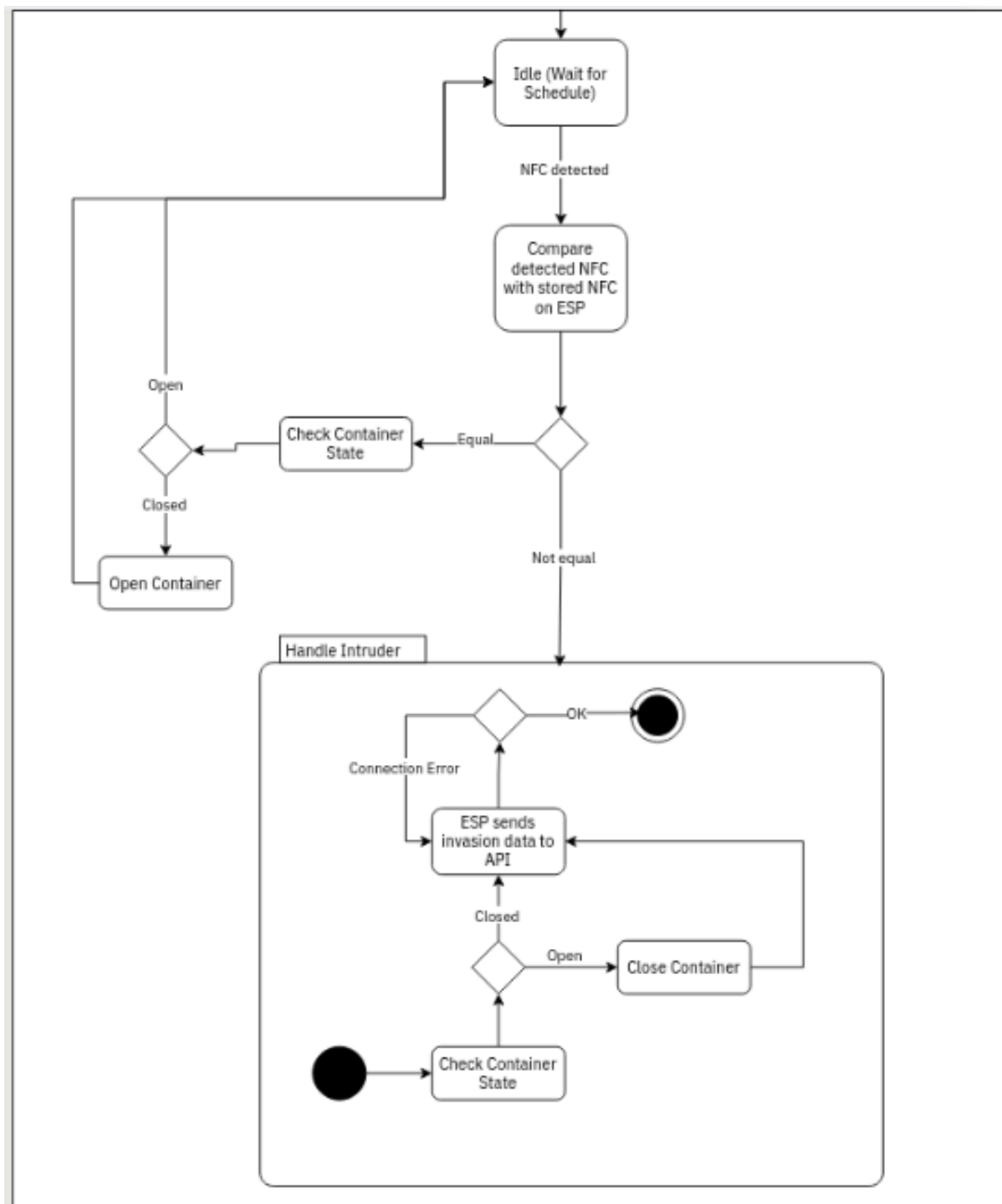


Figura 5: Etapa de detecção de NFC: verificação da tag detectada e envio de alerta de intruso, caso necessário.

Ao detectar uma tag NFC, o sistema compara-a com a tag armazenada. Se a tag corresponder, o estado do pote (aberto/fechado) é verificado e, se necessário, o pote é aberto. Caso a tag não corresponda, o sistema trata o evento como uma intrusão: verifica o estado do pote, fecha-o caso esteja aberto e envia os dados da invasão à API.

5.4.3 Parte 3: Liberação e monitoramento de consumo

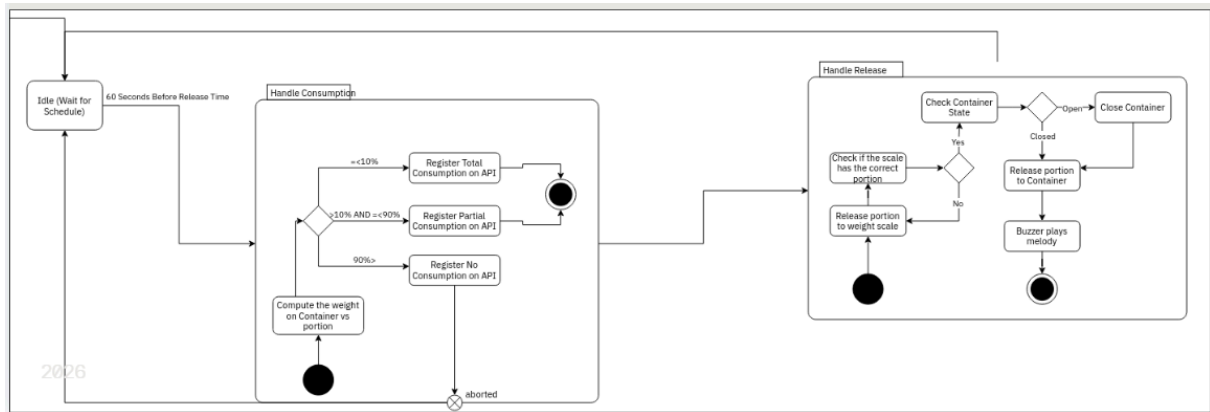


Figura 6: Etapa de liberação de ração: registro do consumo e liberação de nova porção, se necessário.

Sessenta segundos antes do próximo horário de liberação, o sistema calcula o peso do recipiente em relação à porção liberada anteriormente e classifica o consumo (total, parcial ou não realizado), registrando o resultado na API. Em seguida, é verificado se a balança contém a porção correta; caso não, uma nova porção é liberada até o recipiente atingir a quantidade programada. Por fim, o pote é fechado e o buzzer soa uma melodia indicando a conclusão do ciclo.

5.4.4 Parte 4: Atualização de configurações

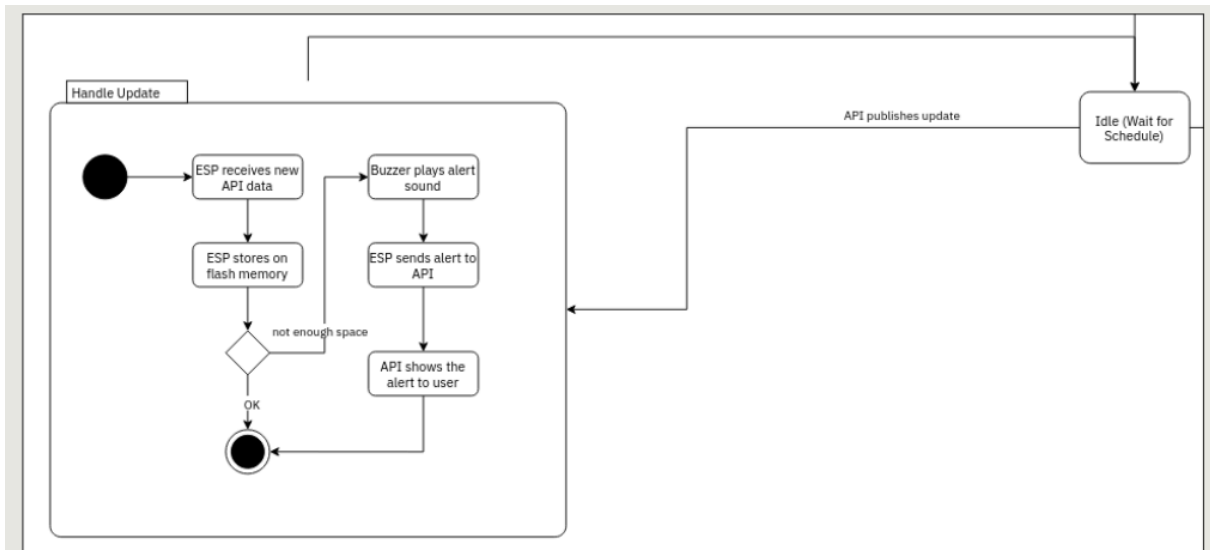


Figura 7: Etapa de atualização do embarcado: recepção de novos dados e persistência na flash, com envio de alertas em caso de falha.

Sempre que o CMS publica uma atualização, o ESP32 recebe os novos dados e os armazena em memória flash. Caso não haja espaço suficiente, o buzzer emite um alerta sonoro e o ESP32 envia um alerta à API, que o exibe ao usuário; caso a gravação seja bem-sucedida, o sistema retorna ao estado de espera (idle).

5.5 Modelo de dados



Figura 8: Modelo físico das entidades do banco de dados (SQLite).

O modelo de dados é composto pelas seguintes entidades principais:

- **User**: usuário responsável pelo cadastro (nome, login, senha);
- **Cat**: dados do gato (nome, data de nascimento, peso, status de castração, identificador NFC), relacionado ao usuário e ao fator de estágio de vida;
- **LifeStageFactor**: fator multiplicador utilizado no cálculo do MER, de acordo com a condição do animal;
- **Ration**: dados da ração cadastrada (descrição, marca, kcal por grama, categoria);
- **Diet**: dieta de um gato, associando o animal a uma ração e à porção calculada;
- **DietSchedule**: horários de alimentação associados a uma dieta;
- **Consumption**: registros de consumo do animal, associados ao tipo de consumo (total, parcial, não realizado) e ao horário de alimentação;
- **ConsumptionTypeId**: tipos de consumo possíveis;
- **IntrusionAlert**: registros de tentativas de acesso por NFC não autorizado.

6 Implementação

6.1 Tecnologias utilizadas

6.1.1 Hardware:

- Microcontrolador ESP8266 (NodeMCU v2) (substituição do ESP32)
- Módulo NFC PN532 V3 / leitor RFID-RC522
- Módulo RTC DS1307 / DS3231 (Real Time Clock, I2C)
- Micro servo motor Tower Pro MG90S (180°) - liberação da ração
- Buzzer passivo 5V
- Elementos de papelão e plástico para representação física do pote e das travas

6.1.2 Software:

- Linguagem C/C++ utilizando Arduino Framework (PlatformIO) para desenvolvimento do sistema embarcado (ESP8266)
- Protocolo MQTT com broker Mosquitto para comunicação entre backend e embarcado
- Protocolo HTTP/REST para comunicação entre aplicativo e backend
- CMS Strapi com banco de dados SQLite
- Aplicação web implementada com Next.js e Material UI

6.2 Tópicos MQTT

6.2.1 CAT/NFC

- Publisher: embarcado
- Subscriber: API
- Objetivo: o embarcado comunicar à API o NFC que foi lido para o novo gato
- Payload:

```
{  
  "nfc": "string"  
}
```

6.2.2 CAT/SYNC

- Publisher: API
- Subscriber: embarcado
- Objetivo: a API conseguir sempre atualizar os dados armazenados no embarcado: NFC, porção em gramas e horários de liberação
- Payload:

```
{  
  "nfc": "string",  
  "portion": 0,  
  "hours": [  
    "00:00",  
    "00:01",  
    // ...  
  ]  
}
```


6.2.3 CAT/INTRUDER

- Publisher: embarcado
- Subscriber: API
- Objetivo: o embarcado comunicar à API todas as tags NFCs detectadas que não sejam as do gato atrelado ao dispositivo, configurando uma invasão
- Payload:

```
{  
  "nfc": "string",  
  "intruder": "string",  
  "date": "iso_date"  
}
```

6.2.4 FEED/REGISTER

- Publisher: embarcado
- Subscriber: API
- Objetivo: o embarcado comunicar à API todos os registros de consumo para que esta classifique e registre posteriormente no banco
- Payload:

```
{  
  "nfc": "string",  
  "amount": number,  
  "date": "iso_date"  
}
```

6.3 Requisitos implementados

6.3.1 RF01 - Cadastro do animal

O sistema realiza o cadastro de um gato em 2 partes: primeiro, o usuário insere a partir da interface dados gerais do gato: nome, data de nascimento, peso, status de castração e quantidade de refeições. Após isso, o ESP8266 realiza a leitura de uma tag NFC e publica no tópico MQTT `cat/nfc`, onde a API lê tal informação e vincula ao gato recém-criado;

6.3.2 RF02 - Geração do plano alimentar

O sistema calcula automaticamente a quantidade diária de ração com base nas características cadastradas do gato, considerando a quantidade de refeições configurada (3, 4 ou 6 por dia), o peso inserido e status de castração.

6.3.3 RF03 - Configuração e distribuição das refeições

O sistema permite configurar 3, 4 ou 6 refeições por dia, com definição manual dos horários de alimentação, e distribui automaticamente a quantidade diária de ração entre os horários configurados.

6.3.4 RF05 - Atualização de dados

O embarcado recebe as novas informações do gato através de uma publicação da API no tópico MQTT `cat/sync`, armazenando as informações atualizadas em variáveis globais e na flash.

6.3.5 RF07 - Identificação NFC

O embarcado, enquanto ligado, realiza a detecção de quaisquer NFCs aproximados, liberando ou bloqueando o pote caso a tag detectada seja inválida.

6.3.6 RF08 - Monitoramento do consumo (Parcial)

O embarcado, antes de fazer a liberação do pote para o gato nos horários das refeições, registra o consumo feito pelo animal e publica no tópico MQTT `feed/register`. Tal consumo foi implementado de forma *mockada*, onde é sempre registrado um consumo completo para fins de exibição na interface. Não foi possível implementar de forma automatizada por problemas de usabilidade e conexão com os sensores de carga HX711.

6.3.7 RF09 - Visualização de estatísticas

O sistema permite a visualização de todos os animais registrados para o usuário, bem como estatísticas para cada um deles que são geradas pelo embarcado ao qual cada gato está vinculado: alertas de intrusão e registros de consumo.

6.3.8 RF10 - Registro de Eventos

O embarcado publica constantemente para a API diversas informações através dos tópicos MQTT, que são interpretadas corretamente pela API e inseridas no banco. As informações consistem nos alertas de intrusão e o registro de consumo.

6.3.9 RNF01 - Persistência local

Os dados persistem na memória flash em caso da rede WiFi estar indisponível.

6.3.10 RNF02 - Confiabilidade após queda de energia

Os dados persistem na memória flash em caso de queda de energia e são todos carregados em formato de variáveis globais.

6.3.11 RNF04 - Tempo de resposta do bloqueio

Ao detectar aproximação indevida, o embarcado fecha o pote em até 1 segundo e imediatamente envia um alerta à API através do tópico MQTT `cat/intruder`.

6.3.12 RNF05 - Comunicação

A comunicação entre App Web e API foi implementada através do protocolo HTTP, enquanto entre API e embarcado foi implementada através do protocolo MQTT, cujos tópicos foram criados através de um broker MQTT Mosquitto local.

6.3.13 RNF06 - Tempo de detecção NFC

A detecção de tags NFC é extremamente ágil e cumpre com o requisito de menos de 1 segundo, desde que o sistema não esteja travado por problemas de conexão como abordado abaixo.

6.4 Limitações

6.4.1 RF04 - Operação offline

O embarcado foi configurado de forma a depender completamente da conexão com o MQTT, que, por sua vez, depende da disponibilidade da rede WiFi. Em caso de problemas com qualquer um dos dois componentes o embarcado pode travar e aguardar pela conexão por tempo indefinido.

6.4.2 RF06 - Liberação e controle da ração (parcial)

A liberação da ração não foi implementada por conflitos de agenda e dificuldades encontradas com a modelagem física da estrutura. Dessa forma, no horário de liberação a única ação executada é a abertura de pote.

6.4.3 RNF03 - Precisão na liberação

Como a liberação não foi implementada, não há critérios aplicáveis de precisão.

6.5 Dificuldades encontradas

6.5.1 Reativação do WiFi/MQTT e risco de travamento

Após reativar WiFi_init, MQTT_init e MQTT_connect no loop principal, foi identificado um problema crítico: em caso de falha na conexão com o broker, a função MQTT_connect() pode entrar em um laço infinito (while(1);), travando completamente o firmware.

Enquanto o MQTT estava desativado, esse comportamento não era observado. Com a reativação, passou a representar um risco direto à operação contínua do sistema, contrariando o requisito de operação offline (RF04). A correção (remoção do bloqueio e adoção de operação degradada com dados locais) ainda não foi implementada.

6.5.2 Integração da célula de carga (HX711)

A integração da célula de carga com o firmware permaneceu pendente. Embora toda a lógica de agendamento, controle de servos e armazenamento local esteja implementada, não foi possível validar a liberação real da ração nem implementar o monitoramento de consumo, pois essas funções dependem da leitura do HX711.

6.5.3 Configuração dinâmica dos horários

Inicialmente o sistema utilizava horários fixos. Para atender ao requisito de flexibilidade, foi implementada uma estrutura dinâmica permitindo 3, 4 ou 6 refeições diárias.

Foi necessário modificar frontend, backend e embarcado. No backend, foi adicionado o atributo feeding_hours ao modelo do gato, além da adaptação da geração do plano alimentar e sincronização via MQTT.

6.5.4 Centralização MQTT no backend

A conexão MQTT foi reorganizada em um módulo compartilhado, permitindo reutilização em diferentes serviços e evitando múltiplas conexões simultâneas.

6.5.5 Configuração do Mosquitto em Docker

Foi necessário ajustar autenticação, permissões e inicialização automática do broker, incluindo criação do arquivo de senhas e configuração de usuários para backend e embarcado.

6.5.6 Orquestração de containers

A comunicação entre serviços exigiu configuração de rede Docker compartilhada, utilizando nomes de serviço como hostname e isolamento de volumes para evitar conflitos de dependências.

6.6 Uso de Inteligência Artificial Generativa

Para o desenvolvimento deste projeto foi utilizada a ajuda da IA generativa nos seguintes pontos:

- *Deteção e correção de bugs*: dada à natureza do projeto e a pouca familiaridade dos integrantes com programação de embarcados, houveram erros capazes de serem detectados somente com a ajuda destas ferramentas;
- *Geração do design dos componentes de UI*: como a interface não é um fim por si só mas parte de um sistema maior cujo principal componente é o embarcado, optamos por construir o esqueleto dos componentes por conta própria e utilizar o auxílio da IA na estilização destes;

7 Conclusão

O desenvolvimento do Xanim's Vault permitiu explorar, de forma prática, os desafios de integração entre um sistema embarcado (ESP8266, em substituição ao ESP32 originalmente planejado), um backend CMS (Strapi) e um broker de mensagens MQTT (Mosquitto), aplicados a um problema real: a alimentação individualizada e segura de felinos em residências com múltiplos animais.

7.1 Resultados alcançados

O protótipo desenvolvido conseguiu validar o núcleo do fluxo proposto e evoluiu em relação à primeira versão, principalmente na flexibilidade da configuração alimentar. O cadastro de um gato no aplicativo (RF01) dispara o cálculo automático do plano alimentar no backend (RF02), agora considerando a quantidade de refeições escolhida pelo usuário entre 3, 4 ou 6 por dia com definição manual dos horários (RF03), o que exigiu mudanças coordenadas no frontend, no backend (novo atributo `feeding_hours` no modelo do gato) e no firmware. Esses dados são sincronizados com o sistema embarcado via MQTT e atualizados localmente sempre que uma nova configuração é recebida (RF05). Toda a estrutura de software para a liberação automática da ração (agendamento e controle dos servos) está implementada (RF06), assim como a identificação por NFC, a validação da tag cadastrada, o bloqueio do pote e o envio de alertas em caso de acesso inválido (RF07). O aplicativo permite visualizar o histórico de alimentação, consumo e tentativas de acesso indevido (RF09), e a estrutura para o registro desses eventos na API também foi implementada (RF10). Os requisitos não funcionais de persistência local (RNF01), integridade dos dados após queda de energia (RNF02), tempo de resposta do bloqueio (RNF04), comunicação via MQTT/HTTP (RNF05) e tempo de detecção do NFC (RNF06) foram atendidos. Toda a stack desde o aplicativo, backend, broker MQTT e embarcado segue orquestrada via Docker, o que manteve o ambiente de desenvolvimento reprodutível entre os integrantes da equipe.

Esses resultados reforçam que a arquitetura proposta é viável e que a divisão de responsabilidades entre os módulos tais como configuração e cálculo da dieta no backend, execução e controle físico no embarcado funciona como esperado nas partes já implementadas, incluindo o ajuste dinâmico de horários que era uma lacuna da versão anterior do projeto.

7.2 Limitações

A limitação mais significativa do protótipo continua sendo a ausência da célula de carga (HX711): embora toda a lógica de agendamento e acionamento dos servos esteja pronta (RF06), não foi possível validar fisicamente a liberação da porção nem implementar o monitoramento de consumo (RF08), que classificaria a refeição como consumo total, parcial ou não realizado a partir do peso restante no prato. Pela mesma razão, a margem de erro de ± 5 g prevista para a liberação (RNF03) não pôde ser garantida. A operação offline também segue como ponto de atenção: o sistema depende do MQTT para atualizar seus dados e a função de reconexão ao broker pode travar indefinidamente o firmware quando a conexão falha, contrariando o requisito de continuar operando com os dados salvos localmente (RF04). Por fim, foi identificado um novo problema durante os testes de NFC: após a primeira leitura de uma tag, o sistema mantém esse estado como “lido” e não reseta a detecção, impedindo que novas aproximações sejam reconhecidas, o que viola o requisito de detecção contínua em até 1 segundo (RNF06).

7.3 Melhorias futuras

A partir do que foi identificado ao longo do desenvolvimento, os próximos passos naturais do projeto seriam:

- Integrar de fato a célula de carga com o HX711, calibrando o fator de conversão para gramas, e assim viabilizar a liberação controlada por peso (RF06), o monitoramento de consumo (RF08) e a margem de ± 5 g da RNF03;
- Corrigir a função de reconexão ao broker MQTT para que, em caso de falha persistente, o firmware continue operando de forma autônoma com os dados salvos na memória flash, em vez de travar indefinidamente (RF04);
- Corrigir o reset do estado de leitura do módulo NFC após cada detecção, permitindo que novas aproximações sejam reconhecidas de forma contínua, conforme previsto na RNF06;
- Validar formalmente o tempo de resposta do bloqueio do pote (RNF04) com testes repetidos e medições, em vez de observação manual via `Serial`;
- Migrar o protótipo para o hardware de referência original (ESP32), avaliando se os conflitos de biblioteca que motivaram o uso do ESP8266 (NodeMCU) se repetem nessa plataforma;
- Substituir os elementos de papelão do protótipo por peças impressas em 3D ou usinadas, aproximando a montagem física do produto final e permitindo testar a vedação do pote e o encaixe dos servomotores em condições mais realistas.

De forma geral, o projeto avançou em relação à sua primeira versão ao resolver a flexibilidade dos horários de alimentação, mas segue dependente da integração do sensor de peso e da robustez da conectividade do firmware para funcionar de ponta a ponta conforme especificado nos requisitos. Uma vez resolvidos esses dois pontos, o Xanim's Vault estaria apto a operar de forma completa e autônoma.

8 Referências

- The 2021 AAHA Nutrition and Weight Management Guidelines for Dogs and Cats.
- Comedouro automático para cães em situação de rua e resgatados por ONGs. Disponível em: <https://joins.emnuvens.com.br/joins/article/view/1026/1069>.
- Alimentador automático para nutrição de gatos com ESP32. UFRGS. Disponível em: <https://lume.ufrgs.br/handle/10183/301126>.
- ComunaPet - Solução inteligente para alimentação de pets através do uso de IoT. Disponível em: <https://releia.ifsertaope.edu.br/jspui/bitstream/123456789/1021/1/TCC%20-%20COMUNAPET%20SOLUCAO%20INTELIGENTE%20PARA%20ALIMENTACAO%20DE%20PETS%20ATRAVES%20DO%20USO%20DE%20IOT.pdf>.
- Dispenser de ração automático com Arduino. Disponível em: http://ric-cps.eastus2.cloudapp.azure.com/bitstream/123456789/20426/1/electronica_2024_1__arthursimionatochiqini_dispenseralimentadorpetautomatico.pdf.
- Alimentador na UFERSA com ESP32 e MQTT e dashboards. Disponível em: <https://repositorio.ufersa.edu.br/server/api/core/bitstreams/862731d1-a70c-4e71-9db0-696525ea7b9b/content>.
- Coleira com tecnologia NFC. Disponível em: <http://ric-cps.eastus2.cloudapp.azure.com/handle/123456789/31918>.